

2025年春季学期哈尔滨工业大学(威海)期末考试题

计算机系统

【声明】

1. 本项目为公益项目,旨在帮助学弟学妹期末备考、或同级学生补考复习使用,请勿拿去售卖。
2. 本试卷为回忆版,不存在窃题漏题等作弊嫌疑.部分数据被遗忘,用编造的数据替代.如认为该题目不应当流出,可以联系「wuwanweihua@gmail.com」,我会及时删除。

一、单项选择题(共10分,在答题框中作答)

1. C 语言带符号数加法溢出的充分必要条件是 ()。
A. 结果小于任一加数
B. 结果大于任一加数
C. 加数同号,结果与其异号
D. 加数同号,结果与其同号
2. 在 x86-64 架构中,当采用对齐存放规则时,一个 double 型数据(8 字节)可能的存放地址为 ()。
A. 0x00007FFFFFFFFFFFFF
B. 0x0000393939393939
C. 0x0000282828282828
D. 0x0000000000000001
3. 发生过程调用时,参数传递一般借助何种寄存器?当需要保存这些寄存器的原有值时,程序会把它们存在哪里? ()
A. caller-saved, 栈区
B. caller-saved, 堆区
C. callee-saved, 栈区
D. callee-saved, 堆区
4. 下列关于文件操作的命令或函数中,说法不正确的是 ()。
A. stat() 为获取文件元数据的系统调用
B. chmod 命令用于修改文件的读、写和(或)执行权限
C. open() 库函数执行成功则返回被打开文件的文件指针
D. fopen() 库函数返回被打开文件的文件指针
5. 链接过程中,被赋了初值的局部变量名是 ()。
A. 强符号
B. 弱符号
C. 若加 static 修饰则为强符号
D. 不是链接符号,无所谓强弱
6. 条件转移指令 JE 依据 () 标志位决定是否跳转。
A. ZF
B. OF
C. SF
D. CF
7. x86-64 处理器使用 () 寄存器指明下一条条件执行指令的地址。
A. %rbx
B. %rbp
C. %rsp
D. %rip
8. 设 %ax 中的原有值为 0xFF70,则执行指令 salw \$2,%ax 后,%ax 的值变为 ()。
A. 0xFDC0
B. 0xFFDC
C. 0xFDC3
D. 0x3FDC
9. x86-64 处理器采用几级页表?其 Cache 块长为多少? ()
A. 单级, 64 位
B. 单级, 64 字节
C. 四级, 64 位
D. 四级, 64 字节
10. 同步异常不包括 ()。
A. 终止
B. 陷阱
C. 停止
D. 故障

二、判断对错(对的画圈错的打叉,共10分)

1. 现代超标量流水 CPU 指令的平均周期接近于 1 个但大于 1 个时钟周期。
2. CPU 硬件层面其实无法判断参与加法运算的是带符号数还是无符号数。

3. 浮点数做除精时取舍的舍入规则是四舍五入。
4. 编译器优化前后的程序运行结果一定不会改变。
5. `switch` 语句采用的跳转表保存在 ELF 文件的 `.rodata` 节。
6. 为防缓存区溢出攻击，可令栈区可读不可执行。
7. 计算机中 `double` 型的浮点数有无穷多。
8. 对于 `unsigned ux`, `ux*ux >= 0` 一定为真。
9. x86-64 的所有寄存器都可以直接赋值。
10. 向进程发信号的函数是 `signal()`，设置值处理程序的函数是 `kill()`。

三、计算与简答（共30分）

1. 采用 5 位二进制表示的带符号数 $x = -13$ 和无符号数 $uy = x$ ，试写出以下表达式的计算结果（二进制补码的形式和对应的数值（十进制表示））。其中 $TMax$ 表示带符号数最大值， $TMin$ 表示带符号数最小值。（5分）
 - (1) uy
 - (2) $x - uy$
 - (3) $TMax + 1$
 - (4) $TMin + 1$
 - (5) $TMax + TMin$
2. IEEE 754 双精度浮点数能够精确表示的最小非零正数是多少？用十六进制表示其编码，并给出对应的十进制真值。（6分）
3. 假设某系统采用单级页表进行虚拟地址到物理地址的映射。已知虚拟地址为 16 位，页大小为 4KB (2^{12} 字节)，物理地址为 18 位，页表条目中包含：1 位有效位以及 6 位物理页号 PPN。部分页表内容如下所示。试计算虚拟地址 0x3A56 对应的物理地址，写出详细计算过程。（5分）

VPN	有效位	PPN
0x2	1	0x1C
0x3	1	0x2D
0x4	0	0x00
0x5	1	0x15

4. 试给出 Vim 的任意 1 条复制命令和 1 条删除命令并解释其含义。例：y\$ 命令，复制从光标所在位置到本行行末之间的全部内容。（4分）



5. 如按段格式在屏幕上列出 `/usr/include` 目录下所有名字以 “std” 开头的头文件（.h 文件）的详细信息，写出对应的 Linux 命令（可从如下选项选取若干元素：`cd rm ls cp -l -a -d x ? | >`）。（5分）
6. 什么是 TLB？为什么需要 TLB？（5分）

四、程序阅读与完形（共10分）

1. 阅读下面的 x86-64 汇编代码，在横线上解释本行指令的功能。（5分）

```
foo:
xorq   %rdx, %rdx           -----
xorq   %rax, %rax           -----
jmp    .L2                  -----
.L3:
addq   (%rdi,%rax,8), %rax   -----
addq   $1, %rdx              -----
.L2:
cmpq   %rsi, %rdx           -----
jl     .L3                  -----
ret
```

2. 上述 foo 过程对应的 C 语言源程序如下，试将其补全。（5分）

```
long foo(long *x, long n)
{
    // 结尾大括号由考生书写
    long val = 0, i;
```

五、分析题（共25分）

1. 阅读下列 C 程序，分析其局部性特征。（5分）

```
int combine(int *a, int n)
{
    volatile int sum = 0;
    int i;

    for (i = 0; i < n; i++)
        sum += a[i];
    return sum;
}
```

- (1) 解释什么时间局部性，什么空间局部性？（2分）
 - (2) 修饰符 volatile 限定 sum 由编译器在内存中维护，说明该程序在读写数据时，空间局部性和时间局部性是如何体现出来的？（4分）
 - (3) 再从取指令的角度，分析该程序为何具有这两种局部性。（4分）
2. 阅读如下 C 程序，分析其输出情况，回答其后的问题。假定：
- (1) 所有进程均正常执行完毕；
 - (2) 所有系统调用均未出错；
 - (3) 所有 printf() 执行均未被打断；
 - (4) 所需头文件包含语句已略去。

```
int main()
{
    pid_t pid1, pid2;
    int a = 10, status;

    pid1 = Fork();
    if (pid1 == 0) {
        a += 5;
        printf("A: %d\n", a);          /* (1) */
        return a;
    } else {
        pid2 = Fork();
        if (pid2 == 0) {
            a -= 3;
            printf("B: %d\n", a);      /* (2) */
            return a;
        } else {
            Waitpid(pid1, &status, 0);
            printf("C: PID1\n");       /* (3) */
            Waitpid(pid2, &status, 0);
            printf("D: PID2\n");       /* (4) */
        }
    }
    return 0;
}
```

- (1) 画出本程序的进程图。(5分)
- (2) 写出本程序所有可能的输出内容。(6分)
- (3) 如果变量 a 添加 static 修饰, 输出结果是否会产生变化? 说明理由。(4分)

六、程序设计与优化 (共15分)

1. 已知某 x86-64 处理器的相关微架构特性如下表所示。在其上执行下面的 C 程序。

操作	延迟	发射
load/store	4	1
整数加	1	1
整数乘	3	1

```
long sum(int a[][N])
{
    long s = 0;

    for (int j = 0; j < N; j++)
        for (int i = 0; i < N; i++)
            s += (a[i][j] * 3);
}
```

```
    return s;  
}
```

- (1) 说明常用的程序优化方法，仅文字叙述即可。(3分)
- (2) 编译器使用 `imul` 指令实现乘“3”操作，试写出一个汇编语言的优化方案，仅写循环体即可，不妨假设 `a[i][j]` 已取至 `%rdx`，`s` 在 `%rax` 中。(3分)
- (3) 写出优化后的 C 语言代码，要求保证语义与原程序等价。(9分)